

ShellGaussian

ShellGaussian: A Unified 3D Representation for Open-Surface Object Modeling

Student Project Canvas

Project Lead: Prof. Feng Liu (VILab, Drexel University)

Target Venue: CVPR 2026

Document Version: 1.0

EXECUTIVE SUMMARY

The One-Sentence Pitch

"We break geometry-appearance degeneracy in open-surface 3D reconstruction through frequency-aware gradient routing and topology-intrinsic appearance regularization."

Why This Is a Big Deal

Current 3D reconstruction methods fail on **mixed-topology objects** like shoes, bags, and furniture that have both closed surfaces (shoe sole) and open surfaces (shoe opening, tongue, lace holes). We solve this by **co-designing** geometry and appearance—making them work together as one unified system instead of two separate pipelines fighting each other.

Expected Impact

- **Technical:** First principled solution to geometry-appearance coupling in open-surface reconstruction
 - **Practical:** Enables product-grade 3D asset creation for e-commerce, AR/VR, and design
 - **Benchmark:** 4,452 shoes × 36 views = largest open-surface reconstruction benchmark
-

PART 1: THE PROBLEM WE'RE SOLVING

1.1 What Is 3D Reconstruction?

3D reconstruction means: **Given images of an object from multiple viewpoints, recover its 3D shape and appearance.**

Input: Multiple photos of a shoe from different angles

Output: A 3D model you can rotate, re-light, and use in applications

This is fundamental to:

- **E-commerce:** Virtual try-on, 3D product viewers
- **AR/VR:** Placing virtual objects in real environments
- **Design:** Creating digital twins of physical products
- **Gaming/Film:** Asset creation for virtual worlds

1.2 Why Is This Hard?

The Core Challenge: Geometry-Appearance Coupling

When you look at a photo, what you see is determined by **two things**:

1. **Geometry:** Where the surface is, what direction it faces (the normal)
2. **Appearance:** What color/texture the surface has, how it reflects light

The problem is: **In the final image, these are multiplied together.**

$$\text{Image} = \text{Geometry} \times \text{Appearance} \times \text{Lighting}$$

This means:

- Wrong geometry + Compensating appearance = Correct image ✓
- Correct geometry + Correct appearance = Correct image ✓

Both solutions produce the **same image**, but only one is correct!

Example: Imagine a shiny spot on a shoe. Is it:

- A specular highlight (appearance) on a flat surface (geometry)?
- Or a bump (geometry) with normal appearance?

From a single photo, you can't tell. And naive training finds **any** solution that matches the image—not the **correct** one.

The Open-Surface Challenge

Most reconstruction methods assume objects are **watertight** (completely closed, like a ball). But real products have:

Surface Type	Examples	Challenge
Open surfaces	Shoe opening, bag opening, sleeves	Has boundaries/edges
Thin structures	Shoe tongue, straps, fabric	Can be seen from both sides
Mixed topology	Shoe = closed sole + open upper	Different parts need different handling

Current methods either:

- Force everything to be closed (creates artifacts)
- Fail to represent boundaries cleanly
- Produce "double layers" where there should be one surface

1.3 What We're Going to Do

We will develop **ShellGaussian**, a reconstruction method that:

1. **Handles open surfaces naturally** using G-Shell representation
2. **Disentangles geometry and appearance** so they don't fight each other
3. **Produces both clean geometry AND high-fidelity appearance**

The key insight is **co-design**: instead of training geometry and appearance separately and hoping they agree, we design them to share structure and constrain each other properly.

PART 2: BACKGROUND KNOWLEDGE

2.1 How 3D is Represented in Deep Learning

There are several ways to represent 3D shapes in neural networks. Understanding these is crucial.

2.1.1 Meshes (Traditional)

A mesh consists of:

- Vertices: Points in 3D space (x, y, z)
- Faces: Triangles connecting vertices

Example: A cube has 8 vertices and 12 triangular faces

Pros:

- Industry standard (used in games, films, CAD)
- Efficient rendering
- Explicit surface

Cons:

- Fixed topology (can't easily change number of holes)
- Hard to optimize with neural networks
- Needs UV mapping for textures

2.1.2 Implicit Functions (Modern)

Instead of explicitly storing surface points, we define a function that tells us whether any point is inside or outside the object.

Signed Distance Function (SDF):

$SDF(x, y, z)$ = signed distance to nearest surface

SDF < 0: Point is INSIDE the object
SDF = 0: Point is ON the surface
SDF > 0: Point is OUTSIDE the object

To get the surface: Find all points where SDF = 0

Why this is powerful:

- Any topology can be represented
- Smooth and continuous
- Easy to optimize with neural networks
- Surface is implicitly defined

How it's implemented:

```
# A neural network that predicts SDF
class SDFNetwork(nn.Module):
    def forward(self, xyz): # xyz is (N, 3) points
        # Through MLP layers...
        return sdf_values # (N, 1) signed distances
```

2.1.3 Neural Radiance Fields (NeRF)

NeRF represents a scene as a function from position + viewing direction to color + density:

NeRF: $(x, y, z, \theta, \phi) \rightarrow (r, g, b, \sigma)$

Where:

- (x, y, z) : 3D position
- (θ, ϕ) : Viewing direction
- (r, g, b) : Color
- σ : Density (how "solid" the point is)

Rendering (Volume Rendering):

To render a pixel:

1. Shoot a ray from camera through pixel
2. Sample points along the ray
3. Query NeRF at each point
4. Accumulate color weighted by density

Color = $\int \sigma(t) \cdot c(t) \cdot \exp(-\int \sigma(s) ds) dt$

Pros: Amazing rendering quality **Cons:** Geometry is "fuzzy" (density, not surface); slow training; hard to extract mesh

2.1.4 3D Gaussian Splatting (3DGS)

The newest and currently most popular approach. Represents a scene as a collection of 3D Gaussians:

Each Gaussian has:

- Position $\mu \in \mathbb{R}^3$: Where is the center?
- Covariance $\Sigma \in \mathbb{R}^{3 \times 3}$: How is it shaped/oriented?
- Color c (or spherical harmonics): What color?
- Opacity $\alpha \in [0,1]$: How transparent?

Rendering (Splatting):

To render a pixel:

1. Project each 3D Gaussian to 2D
2. Sort by depth
3. Blend colors front-to-back using alpha compositing

Much faster than NeRF's ray marching!

Visual intuition:

Imagine thousands of colored, semi-transparent 3D ellipsoids.
When viewed from any angle, they blend together to form the image.

Pros:

- Extremely fast rendering (real-time)
- High quality appearance
- Easy to train

Cons:

- No explicit geometry (can't extract clean mesh)
- "Floaters": Gaussians can drift into empty space
- Hard to edit or manipulate

2.1.5 G-Shell (What We'll Use for Geometry)

G-Shell is designed specifically for **open surfaces**. It uses TWO scalar fields:

Field 1: SDF (Signed Distance Field)

Defines a "shell" surface (like a closed balloon)
 $SDF = 0$ gives you this shell

Field 2: mSDF (Manifold Signed Distance Field)

Defined ON the shell surface
Tells you which parts of the shell are "valid"

$mSDF < 0$: This part of the shell is the actual surface
 $mSDF = 0$: Boundary/edge of the surface
 $mSDF > 0$: This part of the shell should be removed

Intuition:

Imagine a balloon (the shell from SDF).
Now imagine drawing regions on the balloon with a marker (mSDF).
Only the marked regions become the final surface.
The edges of the marked regions become boundaries (like shoe opening).

Why this is perfect for shoes:

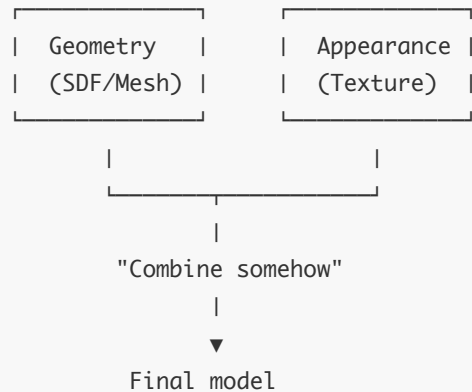
- Shoe sole: SDF defines shape, mSDF marks entire region as valid (closed)
- Shoe opening: SDF defines outer shape, mSDF carves out the opening (boundary at $mSDF=0$)
- Tongue: Can be a thin surface with proper boundary handling

1.3 Our Key Insight: Geometry and Appearance Should Share Structure

The Problem with Separate Representations

Most hybrid approaches treat geometry and appearance separately:

Current approach (problematic):

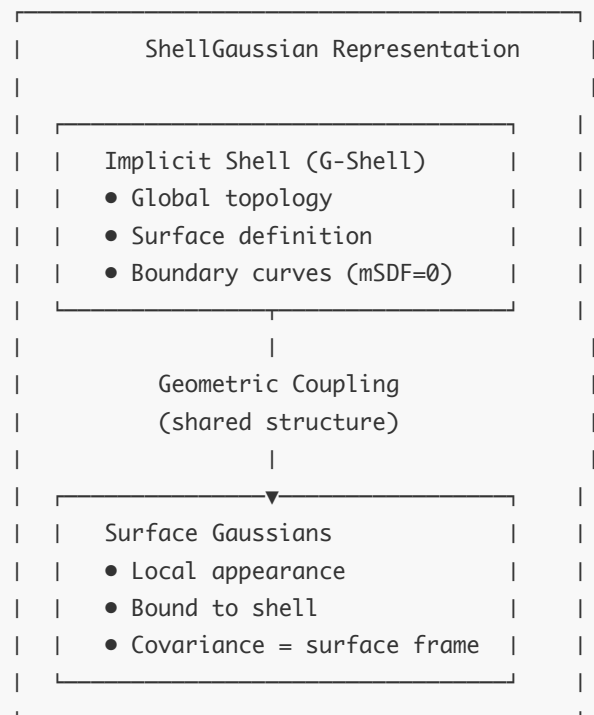


Problems:

- They can disagree
- Optimization fights itself
- Boundaries handled differently
- No shared understanding

Our Approach: Unified Structure

Our approach (unified):



Benefits:

- ✓ Same structure for geometry AND appearance
- ✓ Boundaries handled consistently
- ✓ Optimization is cooperative
- ✓ Principled, not heuristic

2.2 The Rendering Equation (Why Geometry and Appearance are Coupled)

When light hits a surface, the reflected color depends on:

$$L_{\text{out}} = L_{\text{emission}} + \int \text{BRDF}(\omega_{\text{in}}, \omega_{\text{out}}) \times L_{\text{in}}(\omega_{\text{in}}) \times (\mathbf{n} \cdot \omega_{\text{in}}) d\omega$$

Where:

- L_{out} : Light leaving the surface (what we see)
- BRDF: How the surface reflects light (material property)
- L_{in} : Incoming light
- $\mathbf{n}$: Surface normal (GEOMETRY!)
- ω : Light directions

The key observation: The surface normal $\mathbf{n}$ (geometry) is **multiplied** with the BRDF (appearance).

This means:

- If you get the normal wrong, the BRDF can compensate
- If you get the BRDF wrong, the normal can compensate
- Both give the same final image!

This is the degeneracy we need to break.

2.3 Multi-View Reconstruction Pipeline (How Training Works)

```
Training Loop:
1. Initialize 3D representation (random or from structure-from-motion)
2. For each training view:
  a. Render the 3D model from that camera position
  b. Compare rendered image to real photo
  c. Compute loss (difference)
  d. Backpropagate gradients to update 3D representation
3. Repeat until convergence
```

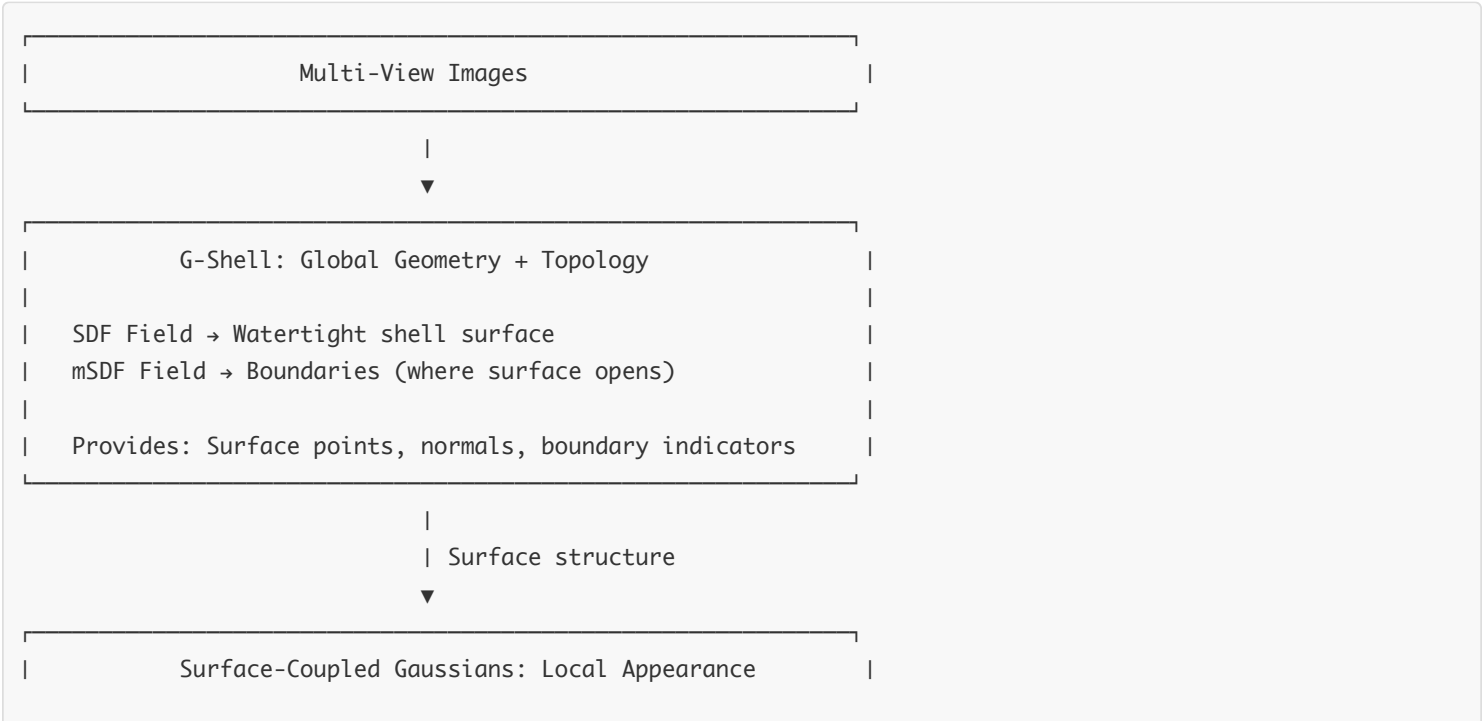
Common losses:

- **Photometric loss:** $||I_{\text{rendered}} - I_{\text{real}}||^2$ (pixel-wise difference)
- **Perceptual loss:** LPIPS (compares deep features, not just pixels)
- **Mask loss:** Silhouette should match
- **Regularization:** Keep surfaces smooth, normals consistent, etc.

PART 3: OUR APPROACH - SHELLGAUSSIAN

3.1 High-Level Idea

We combine **G-Shell** (for geometry) with **3D Gaussian Splatting** (for appearance), but not as two separate systems. Instead, we **co-design** them:



Each Gaussian is BOUND to the G-Shell surface
Covariance aligned with surface tangent space
Color/appearance defined on the surface
Provides: High-fidelity texture, view-dependent effects

|
| Co-design constraints
▼

Co-Designed Training Pipeline
1. Frequency-gated gradient routing
2. mSDF-intrinsic appearance regularization
3. Normal consistency between Gaussians and G-Shell

3.2 Contribution 1: Surface-Coupled Gaussians

The Problem with Regular 3DGS

In standard 3DGS, Gaussians float freely in 3D space:

- They can drift away from actual surfaces ("floaters")
- No connection to geometry
- Can't extract clean mesh

Our Solution: Bind Gaussians to the Surface

Each Gaussian is parameterized relative to the G-Shell surface:

Standard Gaussian:

- Position: $\mu \in \mathbb{R}^3$ (free 3D point)
- Covariance: $\Sigma \in \mathbb{R}^{3 \times 3}$ (arbitrary)

Our Surface-Coupled Gaussian:

- Anchor: u (point on G-Shell surface)
- Offset: δ (small displacement along normal)
- Position: $\mu = \Pi(u) + \delta \cdot n(u)$ where Π maps to surface, n is normal
- Covariance: $\Sigma = R \cdot \text{diag}(\sigma^2_{t1}, \sigma^2_{t2}, \sigma^2_n) \cdot R^T$
where $R = [t_1, t_2, n]$ is the local tangent frame

The Key Insight: Same Σ Encodes Both Geometry and Appearance

The covariance matrix Σ has three eigenvectors and eigenvalues:

- **Smallest eigenvalue direction** → Surface normal (GEOMETRY)
- **Two larger eigenvalue directions** → Tangent plane spread (APPEARANCE extent)

For a surface-like Gaussian:

$\sigma_n \ll \sigma_{t1}, \sigma_{t2}$ (thin in normal direction, spread in tangent plane)

This makes each Gaussian a "surfel" (surface element)

Coupling Constraint: Normal Consistency

$$L_{\text{normal}} = \sum_i || \mathbf{n}_i^{\text{Gaussian}} - \nabla \text{SDF}(\mu_i) / ||\nabla \text{SDF}(\mu_i)|| ||^2$$

This says: The Gaussian's implied normal must match G-Shell's normal

Why this is co-design: Updating the Gaussian's covariance affects geometry (through normal consistency). Updating G-Shell affects Gaussians (through changed surface positions).

3.3 Contribution 2: Frequency-Gated Gradient Routing

The Problem

When you compute image loss (rendered vs. real), the gradient contains BOTH:

- Low-frequency errors → Shape is wrong (silhouette off, depth wrong)
- High-frequency errors → Texture is wrong (details missing, colors off)

If you send ALL gradients to BOTH geometry and appearance:

- Texture errors corrupt geometry
- Specular highlights make geometry "bump" incorrectly
- High-frequency detail tries to be explained by geometry

Our Solution: Route Gradients by Frequency

Step 1: Compute error image

$E = I_{\text{rendered}} - I_{\text{groundtruth}}$

Step 2: Decompose into frequencies (using FFT)

$E_{\text{low}} = \text{InverseFFT}(\text{LowPassFilter}(\text{FFT}(E)))$

$E_{\text{high}} = \text{InverseFFT}(\text{HighPassFilter}(\text{FFT}(E)))$

Step 3: Compute separate losses

$L_{\text{geo}} = ||E_{\text{low}}||^2 \rightarrow$ Only geometry gets gradients

$L_{\text{app}} = ||E_{\text{high}}||^2 \rightarrow$ Only appearance gets gradients

Step 4: Route gradients accordingly

$\partial L_{\text{geo}} / \partial \theta_{\text{geometry}} \rightarrow$ Update G-Shell

$\partial L_{\text{app}} / \partial \theta_{\text{appearance}} \rightarrow$ Update Gaussians' colors

Open-Surface Extension: Boundary-Aware Frequency Shift

Key insight: Boundaries (shoe opening edge) ARE high-frequency geometry!

Near boundaries (where $|\text{mSDF}| < \epsilon$), we expand what geometry "sees":

$$M_{\text{low}}(x) = M_{\text{low_base}} \times (1 + \alpha \times \mathbb{1}[|\text{mSDF}(x)| < \epsilon])$$

Near boundaries: Geometry gets to "see" higher frequencies

Away from boundaries: Normal frequency split

This ensures sharp boundary reconstruction while preventing texture from corrupting interior geometry.

Implementation Sketch

```
def frequency_gated_loss(rendered, target, msdf_map, alpha=0.5, epsilon=0.1):
    # Compute error
    error = rendered - target

    # FFT decomposition
    error_fft = torch.fft.fft2(error)

    # Create frequency masks
    h, w = error.shape[-2:]
    freq_x = torch.fft.fftfreq(w).reshape(1, 1, 1, w)
    freq_y = torch.fft.fftfreq(h).reshape(1, 1, h, 1)
    freq_magnitude = torch.sqrt(freq_x**2 + freq_y**2)

    # Boundary-aware cutoff
    base_cutoff = 0.1 # Tune this
    boundary_mask = (torch.abs(msdf_map) < epsilon).float()
    effective_cutoff = base_cutoff * (1 + alpha * boundary_mask)

    # Apply masks
    low_pass = (freq_magnitude < effective_cutoff).float()
    high_pass = 1 - low_pass

    # Reconstruct
    error_low = torch.fft.ifft2(error_fft * low_pass).real
    error_high = torch.fft.ifft2(error_fft * high_pass).real

    L_geo = (error_low ** 2).mean()
    L_app = (error_high ** 2).mean()

    return L_geo, L_app
```

3.4 Contribution 3: mSDF-Intrinsic Appearance Regularization

The Problem

Standard appearance smoothness uses Euclidean distance:

L_smooth = \sum ||color(x) - color(neighbor_x)||^2

This destroys important appearance transitions at boundaries:

- Shoe interior vs exterior (different colors/materials)
- Tongue edge (may have different texture on each side)
- Stitching along edges

Our Solution: Regularize in mSDF Coordinates

The mSDF field defines a natural coordinate system on the surface:

- $r = |\text{mSDF}|$: Distance to nearest boundary
- θ : Direction along the boundary

Near boundary (small r):

- Allow sharp changes (different inside/outside)
- θ direction follows the edge

Far from boundary (large r):

- Enforce smoothness (interior should be uniform)

The Regularization

L_app_smooth = \int_{\text{Surface}} [||\partial c / \partial r||^2 + (1/r^2) ||\partial c / \partial \theta||^2] dA

Where c = color/appearance

Interpretation:

- $||\partial c / \partial r||^2$: Penalize changes as you move away from boundary
- $(1/r^2) ||\partial c / \partial \theta||^2$: Near boundary (r small), allow angular changes; far from boundary (r large), penalize all changes

Why This Works for Shoes

Region	r value	What happens
Shoe opening edge	$r \approx 0$	Sharp color change allowed (interior vs exterior)
Middle of upper	r large	Smooth texture enforced

Lace holes	$r \approx 0$ at edges	Sharp boundary, smooth interior
Tongue edge	$r \approx 0$	Different colors on each side allowed

Implementation Sketch

```
def msdf_intrinsic_smoothness(colors, positions, msdf_values, msdf_gradients):
    """
    colors: (N, 3) RGB at each surface point
    positions: (N, 3) 3D positions
    msdf_values: (N,) mSDF value at each point
    msdf_gradients: (N, 3) gradient of mSDF (points toward boundary)
    """

    # r = distance to boundary
    r = torch.abs(msdf_values).clamp(min=1e-6)

    # For each point, find neighbors
    # Compute color gradients in r and  $\theta$  directions

    # r direction: along  $-\text{sign}(\text{mSDF}) * \text{msdf\_gradient}$ 
    r_direction = -torch.sign(msdf_values).unsqueeze(-1) * msdf_gradients
    r_direction = r_direction / (torch.norm(r_direction, dim=-1, keepdim=True) + 1e-8)

    #  $\theta$  direction: perpendicular to r_direction on the surface
    # (Need surface tangent plane for this)

    # Compute directional derivatives
    dc_dr = compute_directional_derivative(colors, positions, r_direction)
    dc_dtheta = compute_directional_derivative(colors, positions, theta_direction)

    # Weighted loss
    loss_r = (dc_dr ** 2).mean()
    loss_theta = ((1 / r**2) * dc_dtheta ** 2).mean()

    return loss_r + loss_theta
```

3.5 Putting It All Together: The Training Pipeline

Overall Loss Function

$$L_{\text{total}} = L_{\text{render}} + \lambda_1 \cdot L_{\text{normal}} + \lambda_2 \cdot L_{\text{app_smooth}} + \lambda_3 \cdot L_{\text{vis}} + \lambda_4 \cdot L_{\text{eikonal}}$$

Where:

- L_{render} : Photometric loss (decomposed by frequency)
- L_{normal} : Normal consistency between Gaussians and G-Shell

- L_{app_smooth} : mSDF-intrinsic appearance smoothness
- L_{vis} : Visibility-consistent anti-floater loss
- $L_{eikonal}$: SDF regularization ($\| \nabla SDF \| = 1$)

Training Strategy

Phase 1: Geometry Initialization (warm-up)

- Train G-Shell with silhouette/mask loss
- Get rough shape first

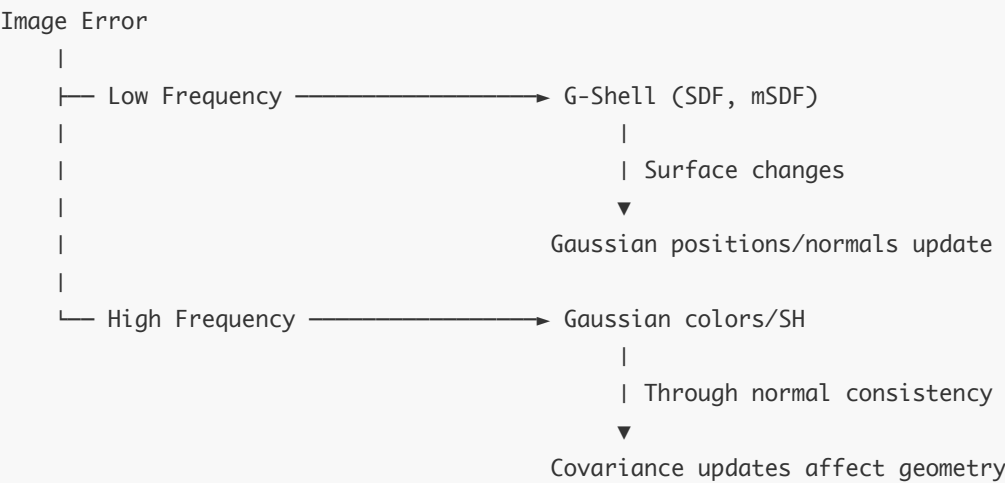
Phase 2: Joint Training

- Add Gaussians on the G-Shell surface
- Train with full loss
- Use frequency-gated gradients

Phase 3: Fine-tuning

- Reduce learning rate
- Focus on boundary quality
- Polish appearance details

Gradient Flow (The Co-Design in Action)



This is the co-design: Changes flow both ways, constrained to be consistent.

PART 4: OUR DATASET

4.1 Overview

Attribute	Value
-----------	-------

Total shoes	4,452 unique products
Total images	~160,000
Views per shoe	36 (360° horizontal rotation)
Vertical variation	Minimal (nearly horizontal ring)
Image quality	High (professional product photography)
Background	Clean/white
Lighting	Consistent studio lighting
Source	StockX product images

4.2 Why This Dataset Is Special

1. **Scale:** 4,452 shoes is much larger than existing object reconstruction datasets
2. **Diversity:** Many shoe types (sneakers, boots, heels, sandals, etc.)
3. **Quality:** Professional photography, not crowd-sourced
4. **Consistency:** Same capture setup across all shoes
5. **Open-surface challenge:** Shoes have openings, tongues, lace holes—perfect for our method

4.3 Data Organization

```
data/
├─ shoes/
│   ├─ product_001/
│   │   ├─ view_000.jpg  (0°)
│   │   ├─ view_001.jpg  (10°)
│   │   └─ ...
│   │   └─ view_035.jpg  (350°)
│   ├─ product_002/
│   │   └─ ...
│   └─ ...
├─ metadata/
│   ├─ camera_poses.json    (estimated or assumed uniform)
│   ├─ categories.json      (sneaker, boot, etc.)
│   └─ splits.json          (train/val/test)
└─ derived/
    ├─ masks/                (foreground segmentation)
    ├─ reconstructions/      (output 3D models)
    └─ features/             (pre-computed features)
```

4.4 Preprocessing Pipeline

Step 1: Image Quality Check

- Remove incomplete sequences
- Remove heavily occluded views
- Flag unusual backgrounds

Step 2: Camera Pose Estimation

- Option A: Assume uniform 10° rotation (quick start)
- Option B: Run COLMAP for accurate poses (better quality)

Step 3: Background Removal

- Run SAM (Segment Anything Model)
- Or use simple color-based segmentation (white background)
- Output: Binary masks for each view

Step 4: Category Annotation (optional but useful)

- CLIP-based zero-shot classification
- Categories: sneaker, boot, sandal, heel, loafer, etc.

Step 5: Train/Val/Test Split

- 80% train, 10% val, 10% test
- Stratified by category

4.5 Copyright Considerations (IMPORTANT!)

- ⚠ DATA SOURCE: StockX product images
- ⚠ COPYRIGHT STATUS: Not cleared for public release

WHAT WE CAN DO:

- ✓ Use for internal research
- ✓ Publish paper with limited example images
- ✓ Release code and model architecture
- ✓ Provide evaluation server (others submit, we evaluate)

WHAT WE CANNOT DO:

- ✗ Publicly release original images
- ✗ Publicly release textured 3D reconstructions
- ✗ Commercial use

MITIGATION STRATEGY:

- Email sent to StockX legal (pending response)
- Consider self-capturing small public subset
- Use evaluation server model for benchmark

PART 5: EXPERIMENTS

5.1 Main Comparisons

Baselines

Method	Type	What It Does	Expected Weakness
3DGS	Pure Gaussian	Fast, high-quality appearance	No geometry; floaters
NeuS2	Pure implicit	Good geometry	Appearance lacks detail
3DGSR	GS + SDF	Hybrid approach	Doesn't handle open surfaces
GSDF	GS + SDF dual	Another hybrid	Same open-surface issues
Neuralangelo	Implicit + hash	High-quality geometry	Slow; appearance limited

Metrics

Novel View Synthesis (Appearance Quality):

PSNR: Peak Signal-to-Noise Ratio (higher is better)

SSIM: Structural Similarity (higher is better)

LPIPS: Learned Perceptual Image Patch Similarity (lower is better)

Geometry Quality:

If ground truth mesh available:

- Chamfer Distance (lower is better)
- F-score (higher is better)
- Normal Consistency (higher is better)

If no ground truth:

- Silhouette IoU: How well does rendered silhouette match input masks?
- Depth Consistency: Is depth consistent across views?

Open-Surface Specific (Our Novel Metrics):

- Boundary IoU:
- Project $mSDF=0$ curves to each view
 - Compare to actual mask boundaries
 - Measures how well we capture edges
- Boundary Sharpness:
- Gradient magnitude at predicted boundaries
 - Higher = sharper edges
- Single-Layer Score:
- Detect double-surface artifacts
 - Sample points near predicted boundaries
 - Check if there's only one surface (not two)

5.2 Ablation Studies

Ablation	What We Remove	Expected Result
w/o Frequency Gating	Use all gradients for all parameters	Geometry corrupted by texture/specular
w/o mSDF Regularization	Standard Euclidean smoothness	Boundaries over-smoothed
w/o Normal Consistency	Gaussians not coupled to surface	More floaters; geometry-appearance mismatch
w/o Boundary-Aware Threshold	Same frequency cutoff everywhere	Blurry boundaries
w/o Surface Coupling	Free Gaussians (standard 3DGS)	Floaters; no clean geometry

5.3 Qualitative Experiments

What to Show in Figures

1. Boundary Quality
 - Close-up on shoe opening
 - Compare: Ours (sharp, single-layer) vs Baselines (blurry, double-layer)
1. Appearance Detail
 - Zoom on texture (leather grain, stitching, logos)
 - Compare: Ours (crisp) vs Pure implicit (blurry)

1. Geometry Mesh

- Extracted mesh visualization
- Compare: Ours (clean, usable) vs 3DGS (no mesh) vs Others (artifacts)

1. Challenging Cases

- Mesh fabric (see-through holes)
- Thin straps (sandals)
- Complex topology (boots with many openings)

Demo Video

- 360° rotation of reconstruction
- Side-by-side with ground truth
- Multiple shoe types
- Zoom on boundaries

5.4 Experimental Protocol

For each shoe in test set:

1. Provide N views (e.g., 24 of 36) for reconstruction
2. Hold out K views (e.g., 12 of 36) for evaluation
3. Reconstruct using each method
4. Evaluate metrics on held-out views
5. Evaluate geometry quality

Report:

- Mean and std across test set
- Per-category breakdown
- Failure case analysis

PART 6: IMPLEMENTATION ROADMAP

6.1 Timeline Overview (24 weeks to submission)

Weeks 1-4: Environment setup + Data preparation
Weeks 5-8: Baseline implementation + Initial experiments
Weeks 9-14: Core method implementation
Weeks 15-18: Full experiments + Ablations
Weeks 19-22: Paper writing + Figures
Weeks 23-24: Buffer + Submission

6.2 Phase 1: Setup and Data (Weeks 1-4)

Week 1: Environment Setup

- ☐ Set up development environment (PyTorch, CUDA)
- ☐ Clone and test existing codebases (3DGS, NeuS2, G-Shell)
- ☐ Set up experiment tracking (wandb/tensorboard)
- ☐ Set up version control (git)

Week 2: Data Download and Organization

- ☐ Download all shoe images
- ☐ Organize by product ID
- ☐ Verify image completeness
- ☐ Basic data loading code

Week 3: Data Preprocessing

- ☐ Background removal/segmentation
- ☐ Camera pose estimation (assume uniform or run COLMAP)
- ☐ Quality filtering

Week 4: Data Visualization

- ☐ Multi-view visualization tools
- ☐ Mask overlay visualization
- ☐ Train/val/test split creation

Deliverable: Clean dataset ready for experiments

6.3 Phase 2: Baseline Understanding (Weeks 5-8)

Week 5-6: Run Existing Methods

- ☐ Run 3DGS on 10 shoes
- ☐ Run NeuS2 on 10 shoes
- ☐ Analyze results: What works? What fails?
- ☐ Document failure modes

Week 7-8: Implement Evaluation

- ☐ Novel view synthesis metrics (PSNR, SSIM, LPIPS)
- ☐ Geometry metrics (if GT available) or silhouette IoU
- ☐ Visualization pipeline for comparisons
- ☐ Create evaluation scripts

Deliverable: Working baseline with evaluation; understanding of failure modes

6.4 Phase 3: Core Method (Weeks 9-14)

Week 9-10: G-Shell Implementation

- ☐ Implement or adapt G-Shell code
- ☐ Train G-Shell on shoes (geometry only)
- ☐ Verify mSDF correctly captures boundaries
- ☐ Debug and tune

Week 11-12: Surface-Coupled Gaussians

- ☐ Implement Gaussian anchoring to surface
- ☐ Implement tangent-frame covariance
- ☐ Implement normal consistency loss
- ☐ Test on simple cases

Week 13-14: Co-Design Training

- ☐ Implement frequency-gated gradient routing
- ☐ Implement mSDF-intrinsic regularization
- ☐ Full training pipeline
- ☐ Initial results on validation set

Deliverable: Working ShellGaussian implementation

6.5 Phase 4: Experiments (Weeks 15-18)

Week 15-16: Main Experiments

- ☐ Full comparison with baselines
- ☐ Run on entire test set
- ☐ Compute all metrics
- ☐ Generate comparison figures

Week 17-18: Ablations and Analysis

- ☐ All ablation experiments
- ☐ Per-category analysis
- ☐ Failure case analysis
- ☐ Hyperparameter sensitivity

Deliverable: All experimental results

6.6 Phase 5: Paper (Weeks 19-22)

Week 19-20: Writing

- [] Introduction draft
- [] Method section
- [] Experiments section
- [] Related work

Week 21-22: Figures and Polish

- [] Create all figures
- [] Supplementary material
- [] Video demo
- [] Internal review and revision

Deliverable: Complete paper draft

6.7 Phase 6: Submission (Weeks 23-24)

Week 23-24: Final Polish

- [] Address review feedback
 - [] Final proofreading
 - [] Format check
 - [] Submit!
-

PART 7: CODEBASE STRUCTURE

7.1 Recommended Organization

```
shellgaussian/  
├─ README.md  
├─ requirements.txt  
├─ setup.py  
├─  
├─ configs/  
│   ├── base.yaml           # Default hyperparameters  
│   ├── shoes.yaml          # Dataset-specific config  
│   └─ ablations/           # Ablation configs  
├─  
├─ data/  
│   ├── __init__.py  
│   ├── dataset.py          # Data loading  
│   ├── preprocessing.py    # Image preprocessing  
│   └─ camera.py            # Camera pose handling  
├─  
├─ models/  
│   └─ __init__.py
```

```

|   ├── gshell/
|   |   ├── sdf_network.py      # SDF prediction network
|   |   ├── msdf_network.py    # mSDF prediction network
|   |   └── extraction.py      # Mesh extraction
|   ├── gaussians/
|   |   ├── gaussian.py        # Surface-coupled Gaussians
|   |   ├── covariance.py      # Tangent-frame covariance
|   |   └── densification.py    # Adaptive Gaussian control
|   └── renderer/
|       ├── rasterizer.py      # Gaussian rasterization
|       └── volume_render.py    # (if needed for comparison)
|
├── losses/
|   ├── __init__.py
|   ├── photometric.py         # Image reconstruction loss
|   ├── frequency_gate.py      # Frequency-gated losses
|   ├── msdf_smooth.py         # mSDF-intrinsic regularization
|   ├── normal_consistency.py  # Gaussian-surface coupling
|   └── geometric.py           # Eikonal, etc.
|
├── training/
|   ├── __init__.py
|   ├── trainer.py             # Main training loop
|   ├── optimizer.py           # Optimizer setup
|   └── scheduler.py           # Learning rate scheduling
|
├── evaluation/
|   ├── __init__.py
|   ├── metrics.py             # PSNR, SSIM, LPIPS, etc.
|   ├── geometry.py            # Geometry metrics
|   └── boundary.py            # Open-surface metrics
|
├── utils/
|   ├── __init__.py
|   ├── visualization.py       # Plotting and rendering
|   ├── io.py                  # Save/load utilities
|   └── logging.py             # Experiment tracking
|
├── scripts/
|   ├── preprocess_data.py     # Data preprocessing
|   ├── train.py               # Training script
|   ├── evaluate.py            # Evaluation script
|   └── visualize.py           # Visualization script
|
└── experiments/
    ├── run_baseline.sh
    ├── run_ablations.sh
    └── run_full.sh

```

7.2 Key Dependencies

```

# requirements.txt

# Deep Learning
torch>=2.0.0
torchvision>=0.15.0
pytorch3d>=0.7.0          # 3D operations

# 3D Processing
trimesh>=3.9.0            # Mesh I/O
open3d>=0.17.0            # Point cloud processing
kaolin>=0.14.0            # NVIDIA 3D library (optional)

# Gaussian Splatting
diff-gaussian-rasterization # From original 3DGS repo
simple-knn                  # From original 3DGS repo

# Image Processing
opencv-python>=4.5.0
pillow>=9.0.0
scikit-image>=0.19.0

# Metrics
lpips>=0.1.4              # Perceptual loss
pytorch-fid                # FID computation

# Utilities
numpy>=1.21.0
scipy>=1.7.0
matplotlib>=3.5.0
tqdm>=4.60.0
omegaconf>=2.1.0          # Config management
wandb>=0.12.0             # Experiment tracking

# Segmentation (for preprocessing)
segment-anything           # SAM

```

7.3 Getting Started Code

Basic Data Loading

```

# data/dataset.py

import torch
from torch.utils.data import Dataset
from pathlib import Path
import cv2
import json

class ShoeDataset(Dataset):
    def __init__(self, root_dir, split='train', num_views=36):

```



```

self.root = Path(root_dir)
self.split = split
self.num_views = num_views

# Load split info
with open(self.root / 'metadata' / 'splits.json') as f:
    splits = json.load(f)
self.product_ids = splits[split]

# Load camera poses
with open(self.root / 'metadata' / 'camera_poses.json') as f:
    self.poses = json.load(f)

def __len__(self):
    return len(self.product_ids)

def __getitem__(self, idx):
    product_id = self.product_ids[idx]
    product_dir = self.root / 'shoes' / product_id

    images = []
    masks = []
    poses = []

    for view_idx in range(self.num_views):
        # Load image
        img_path = product_dir / f'view_{view_idx:03d}.jpg'
        img = cv2.imread(str(img_path))
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        img = torch.from_numpy(img).float() / 255.0
        images.append(img)

        # Load mask
        mask_path = self.root / 'derived' / 'masks' / product_id / f'view_{view_idx:03d}.png'
        if mask_path.exists():
            mask = cv2.imread(str(mask_path), cv2.IMREAD_GRAYSCALE)
            mask = torch.from_numpy(mask).float() / 255.0
        else:
            mask = torch.ones(img.shape[:2])
        masks.append(mask)

        # Get pose
        pose = torch.tensor(self.poses[product_id][view_idx])
        poses.append(pose)

    return {
        'product_id': product_id,
        'images': torch.stack(images),      # (N, H, W, 3)
        'masks': torch.stack(masks),       # (N, H, W)
        'poses': torch.stack(poses),       # (N, 4, 4)
    }

```

Frequency-Gated Loss

```
# losses/frequency_gate.py

import torch
import torch.nn.functional as F

class FrequencyGatedLoss:
    def __init__(self, base_cutoff=0.1, boundary_alpha=0.5, boundary_epsilon=0.1):
        self.base_cutoff = base_cutoff
        self.boundary_alpha = boundary_alpha
        self.boundary_epsilon = boundary_epsilon

    def __call__(self, rendered, target, msdf_map=None):
        """
        Args:
            rendered: (B, H, W, 3) rendered images
            target: (B, H, W, 3) target images
            msdf_map: (B, H, W) projected mSDF values (optional)

        Returns:
            L_geo: Loss for geometry updates
            L_app: Loss for appearance updates
        """
        # Compute error
        error = rendered - target # (B, H, W, 3)

        # Move to frequency domain
        error_fft = torch.fft.rfft2(error.permute(0, 3, 1, 2)) # (B, 3, H, W//2+1)

        # Create frequency grid
        B, C, H, W_freq = error_fft.shape
        freq_y = torch.fft.fftfreq(H, device=error.device).view(1, 1, H, 1)
        freq_x = torch.fft.rfftfreq(W_freq * 2 - 1, device=error.device).view(1, 1, 1, W_freq)
        freq_magnitude = torch.sqrt(freq_y**2 + freq_x**2)

        # Compute cutoff (boundary-aware if msdf provided)
        if msdf_map is not None:
            # Need to handle spatial variation - simplified version
            boundary_mask = (torch.abs(msdf_map) < self.boundary_epsilon).float()
            avg_boundary = boundary_mask.mean(dim=[1, 2], keepdim=True) # Per-image
            cutoff = self.base_cutoff * (1 + self.boundary_alpha * avg_boundary)
            cutoff = cutoff.view(B, 1, 1, 1)
        else:
            cutoff = self.base_cutoff

        # Create masks
        low_pass = (freq_magnitude < cutoff).float()
        high_pass = 1.0 - low_pass

        # Apply masks and inverse FFT
```

```
error_low_fft = error_fft * low_pass
error_high_fft = error_fft * high_pass

error_low = torch.fft.irfft2(error_low_fft, s=(H, rendered.shape[2]))
error_high = torch.fft.irfft2(error_high_fft, s=(H, rendered.shape[2]))

# Compute losses
L_geo = (error_low ** 2).mean()
L_app = (error_high ** 2).mean()

return L_geo, L_app
```

PART 8: READING LIST

8.1 Must-Read Papers (Start Here)

Core Technical Background

Paper	Year	Why Read It
NeRF: Representing Scenes as Neural Radiance Fields	2020	Foundation of neural rendering
NeuS: Learning Neural Implicit Surfaces	2021	Implicit surfaces from images
3D Gaussian Splatting for Real-Time Radiance Field Rendering	2023	Our appearance backbone
G-Shell: Ghost on the Shell	2024	Our geometry backbone

Relevant Hybrid Methods

Paper	Year	Why Read It
3DGSR: 3D Gaussian Splatting for Surface Reconstruction	2024	Direct competitor
GSDF: 3DGS Meets SDF for Improved Rendering and Reconstruction	2024	Another hybrid approach

SuGaR: Surface-Aligned Gaussian Splatting	2024	Surface-aware Gaussians
NeuSG: Neural Implicit Surface Reconstruction with 3D Gaussian Splatting	2024	Related approach

8.2 Helpful Background

Differentiable Rendering

- Differentiable Rendering: A Survey (2022)
- PyTorch3D documentation

Multi-View Reconstruction

- COLMAP paper and documentation
- MVSNet series (for depth-based MVS)

Loss Functions and Training

- Perceptual Losses for Real-Time Style Transfer (LPIPS origins)
- Curriculum Learning (for training strategies)

8.3 Reading Strategy

- Week 1-2: Core papers
- Read NeRF carefully (understand volume rendering)
 - Read 3DGS carefully (understand Gaussian rasterization)
 - Skim NeuS and G-Shell (understand SDF and mSDF)
- Week 3-4: Run code
- Clone and run 3DGS on example data
 - Clone and run NeuS2 if possible
 - Understand the codebase structure
- Week 5-6: Related work
- Read hybrid methods (3DGSR, GSDF, SuGaR)
 - Identify their limitations
 - Understand how we differ

PART 9: COMMON PITFALLS AND TIPS

9.1 Technical Pitfalls

Pitfall 1: G-Shell Training Instability

Problem: SDF/mSDF fields collapse or produce weird shapes

Solutions:

- Initialize with sphere SDF
- Use eikonal regularization: $\|\nabla \text{SDF}\| = 1$
- Start with coarse grid, progressively refine
- Check for NaN gradients

Pitfall 2: Gaussians Floating Away

Problem: Gaussians drift from surface (floaters)

Solutions:

- Enforce surface binding loss: $L = \sum |\text{SDF}(\mu_i)|^2$
- Limit maximum offset δ
- Aggressive pruning of distant Gaussians
- Visibility-based pruning

Pitfall 3: Frequency Gating Too Aggressive

Problem: Geometry becomes over-smoothed or appearance loses detail

Solutions:

- Tune cutoff frequency carefully
- Use soft masks instead of hard cutoff
- Monitor both losses during training
- Ablate to find right balance

Pitfall 4: mSDF Boundaries Not Sharp

Problem: Open surface boundaries are blurry

Solutions:

- Increase boundary region detection (ϵ)
- Add explicit edge loss (align with mask boundaries)
- Use higher resolution for mSDF near boundaries
- Check mSDF gradient direction

Pitfall 5: Training Too Slow

Problem: Each shoe takes hours to reconstruct

Solutions:

- Use hash encoding for faster SDF queries
- Reduce number of Gaussians initially

- Coarse-to-fine training strategy
- Profile code to find bottlenecks

9.2 Research Tips

Tip 1: Start Simple

Don't implement everything at once.
Start with: G-Shell only → Add Gaussians → Add frequency gating → Add mSDF regularization
Validate each step before moving on.

Tip 2: Visualize Everything

Create visualizations for:

- SDF isosurface
- mSDF on surface
- Gaussian positions and orientations
- Frequency decomposition of errors
- Gradient magnitudes

Bugs are often visible before they show in metrics.

Tip 3: Test on Synthetic Data First

Before running on real shoes:

1. Create synthetic data (render known 3D models)
2. Test reconstruction (you know ground truth)
3. Debug on simple shapes (sphere with hole)
4. Then move to real data

Tip 4: Keep a Research Log

Daily log:

- What I tried
- What worked / didn't work
- Hypotheses for why
- Next steps

This is invaluable for paper writing and debugging.

Tip 5: Communicate Progress

Weekly meeting with advisor:

- Show visualizations (not just numbers)
- Discuss blockers

- Get feedback early
- Don't wait until something is "perfect"

PART 10: SUCCESS CRITERIA

10.1 For Paper Acceptance

Criterion	Minimum	Target (for Oral)
PSNR	+1dB over best baseline	+2dB over best baseline
Boundary IoU	+5% over baselines	+10% over baselines
Qualitative	Visible improvement in figures	Dramatically better boundaries
Ablations	All three contributions show gains	Clear, large gains for each
Dataset	Use full 4K shoes	+ Per-category analysis

10.2 Milestone Checkpoints

Checkpoint 1 (Week 4)

- ☐ Dataset preprocessed and loading correctly
- ☐ Can run baseline (3DGS) on one shoe
- ☐ Visualization pipeline working

Checkpoint 2 (Week 8)

- ☐ Baselines running on 100+ shoes
- ☐ Evaluation metrics implemented
- ☐ Understand failure modes

Checkpoint 3 (Week 14)

- ☐ ShellGaussian implementation complete
- ☐ Initial results on validation set
- ☐ Results look promising (qualitatively)

Checkpoint 4 (Week 18)

- [] All experiments complete
- [] Ablations show each contribution helps
- [] Figures ready for paper

Checkpoint 5 (Week 22)

- [] Paper draft complete
- [] Supplementary and video ready
- [] Internal review feedback addressed

10.3 Red Flags (When to Seek Help)

- Results not improving after 2 weeks of trying
- Unexplained NaN or divergence
- Running out of GPU memory
- Unclear how to implement a component
- Confused about a paper/concept

Don't wait too long to ask for help!

APPENDIX A: KEY EQUATIONS REFERENCE

A.1 G-Shell Surface Extraction

$SDF(x) = 0$	→ Shell surface
$mSDF(x) < 0$ on shell	→ Valid surface region
$mSDF(x) = 0$ on shell	→ Boundary curves

A.2 Surface-Coupled Gaussian

Position: $\mu_i = \Pi(u_i) + \delta_i \cdot n(u_i)$

Covariance: $\Sigma_i = R_i \cdot \text{diag}(\sigma^2_{t1}, \sigma^2_{t2}, \sigma^2_n) \cdot R_i^T$
 where $R_i = [t_1, t_2, n]$ is tangent frame

Constraint: $\sigma_n \ll \sigma_{t1}, \sigma_{t2}$ (thin in normal direction)

A.3 Frequency-Gated Loss

$E = I_{\text{rendered}} - I_{\text{target}}$

$L_{\text{geo}} = ||\mathcal{F}^{-1}[M_{\text{low}} \cdot \mathcal{F}(E)]||^2$

$L_{\text{app}} = ||\mathcal{F}^{-1}[M_{\text{high}} \cdot \mathcal{F}(E)]||^2$

Boundary-aware cutoff:
 $M_{\text{low}}(x) = M_{\text{low_base}} \times (1 + \alpha \times \mathbb{1}[|mSDF(x)| < \epsilon])$

A.4 mSDF-Intrinsic Regularization

Coordinates:
 $r = |mSDF|$ (distance to boundary)
 $\theta = \text{along boundary}$ (tangent direction)

Loss:
 $L_{\text{smooth}} = \int_S [||\partial c / \partial r||^2 + (1/r^2) ||\partial c / \partial \theta||^2] dA$

A.5 Normal Consistency

$$L_{\text{normal}} = \sum_i ||n_i^{\text{Gaussian}} - \nabla SDF(\mu_i) / ||\nabla SDF(\mu_i)|| ||^2$$

A.6 Total Loss

$L = L_{\text{render}} + \lambda_1 \cdot L_{\text{normal}} + \lambda_2 \cdot L_{\text{smooth}} + \lambda_3 \cdot L_{\text{eikonal}} + \lambda_4 \cdot L_{\text{vis}}$

Where:
 $L_{\text{render}} = L_{\text{geo}} + L_{\text{app}}$ (frequency-decomposed)
 $L_{\text{eikonal}} = (||\nabla SDF|| - 1)^2$ (SDF regularization)
 $L_{\text{vis}} = \text{visibility-consistent anti-floater loss}$

APPENDIX B: GLOSSARY

Term	Definition
SDF	Signed Distance Function - gives signed distance to surface
mSDF	Manifold SDF - defined on shell surface, marks valid regions
3DGS	3D Gaussian Splatting - representing scenes with Gaussians
G-Shell	Geometry representation using SDF + mSDF for open surfaces

Covariance	Matrix describing Gaussian's shape and orientation
Tangent frame	Local coordinate system on surface (two tangents + normal)
Floaters	Gaussians that drift away from actual surface
Eikonal	Property that
PSNR	Peak Signal-to-Noise Ratio (image quality metric)
LPIPS	Learned Perceptual Image Patch Similarity
Novel view synthesis	Generating images from viewpoints not in training set
Multi-view	Multiple images of same object from different viewpoints
Topology	Connectivity structure (open vs closed, number of holes)